ILAB CAPSTONE · Group 13-01

Next-Category Prediction & Smart Documentation for MyVal

AGENDA



1 Motivation & Objective

The under-documentation problem, our target, and what we built



2 Related Work & Background

What already existed, what this project added, leakage controls



3 Our Approach

Data, architecture, model design, the two-stage loop, the stop limit



4 Main Results & Demo

Classifier and regression results, replay evidence, the real app



5 Discussion & Evaluation

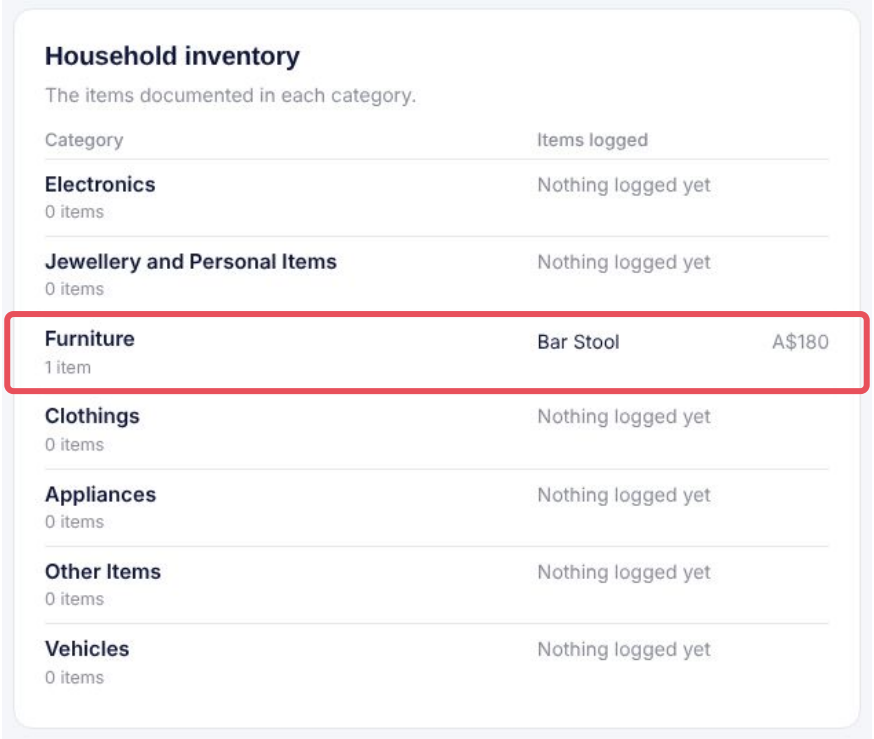
What's verified, what's a simple rule, and where the evidence is thin



6 Conclusion & Reflection

Technical contributions and defensible next steps

Incomplete household inventories



Our prototype, running on an example household (PRP-000059) from our dataset

Example: 1 item logged

A bar stool worth A$180.
Every other category: nothing logged yet.

An incomplete inventory can mean incomplete insurance cover.

THE QUESTION

How can we help the customer remember what to document next?

Our solution: predict, suggest, stop

Objective: given what a household has already documented, predict the next previously undocumented category worth asking the customer to capture.

1 Predict

Which category should the customer document next?

POWERED BY
Category classifier

2 Suggest

Ask one question, with a value the customer can edit.

POWERED BY
Rule-based median value

3 Stop

Estimate when more suggestions may stop being useful.

POWERED BY
Household-value regression

A heuristic. Applies once 10+ valued items are logged.

All of it runs in one FastAPI + browser app, and refreshes as the customer logs items.

Built on 234 properties (431 prediction states) from a synthetic dataset. The data and models are covered next.

What Exists, Prior Research and What This Project Added

What exists in myVal

The data in the app: properties, logged items, categories and values.

No ML models or recommender existed.

What this project added

1. Models: a next-category classifier (two Random Forests) and a household contents-value regression (XGBoost + LightGBM)

2. Recommender system: a rule-based suggestion loop that uses the classifier's ranking to ask one question at a time, served by a small web app.

Prior research on recommenders

Recommenders suggest what a user is likely to want: Adomavicius & Tuzhilin (2005).

Regression predicts a score (Koren et al., 2009); classification predicts an action (He et al., 2014).

Hybrids combine models with rules: Burke (2002).

How leakage is controlled

- Documented-asset features only use assets created by the prediction time; the target category is never already documented in its own state (0 of 431)
- Preprocessing is fitted on training data only, and train / validation / test are grouped by Property_ID - zero property overlap between partitions
- The final test set stayed untouched during model selection and tuning

Partition	Rows	Properties
Train	309	163
Validation	67	35
Test	55	36

A known limit we flag openly: the AI-analysis and video features are not strictly point-in-time, and 30 customers own more than one property (7 of 55 test states belong to a customer also in train). A reviewer's refit without those feature groups still scored 0.43–0.45, close to the full model - a supplementary check, not an audited notebook result.

One row per household state, five feature groups

Dataset : Consisted of 24 sheets , out of which 6 were relevant to our problem statement.

Target Classifier: Next_Category, the next not-yet-documented contents category (one of 7, Vehicles is effectively never suggested).
Target Regressor: Estimate , weighted average sum total of all assets of a household.

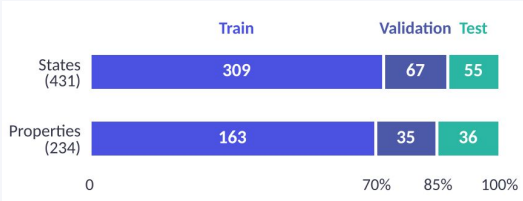
Feature group	Example columns	Point-in-time?
Documented assets	documented_count_CAT-00x, documented_value_total, documented_replacement_cost_total, evidence rating	● Yes, by construction
AI analysis	ai_count_CAT-00x, ai_estimated_value_total / _mean / _max, ai_suggestion_acceptance_rate, ai_category_match_rate	● Not strictly (see limits)
Video-derived	Property-video counts, summed over the whole property: total_rooms_covered, video_count	● Not strictly (see limits)
Customer	Tenure_Months, Financial_Literacy_Band, Membership_Level, Engagement_Tier	● Static snapshot
Property	Property_Type, Property_Valuation_AUD, Number_Of_Occupants, Number_Of_Bedrooms	● Static snapshot

From workbook to model



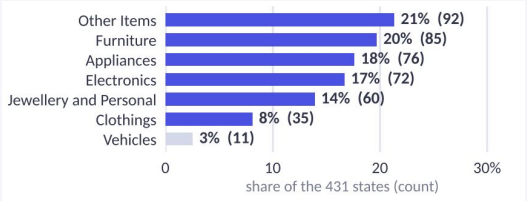
55% of properties (440 of 795) document only one category. 2 states had no AI output and were dropped.

Split by property, 70 / 15 / 15



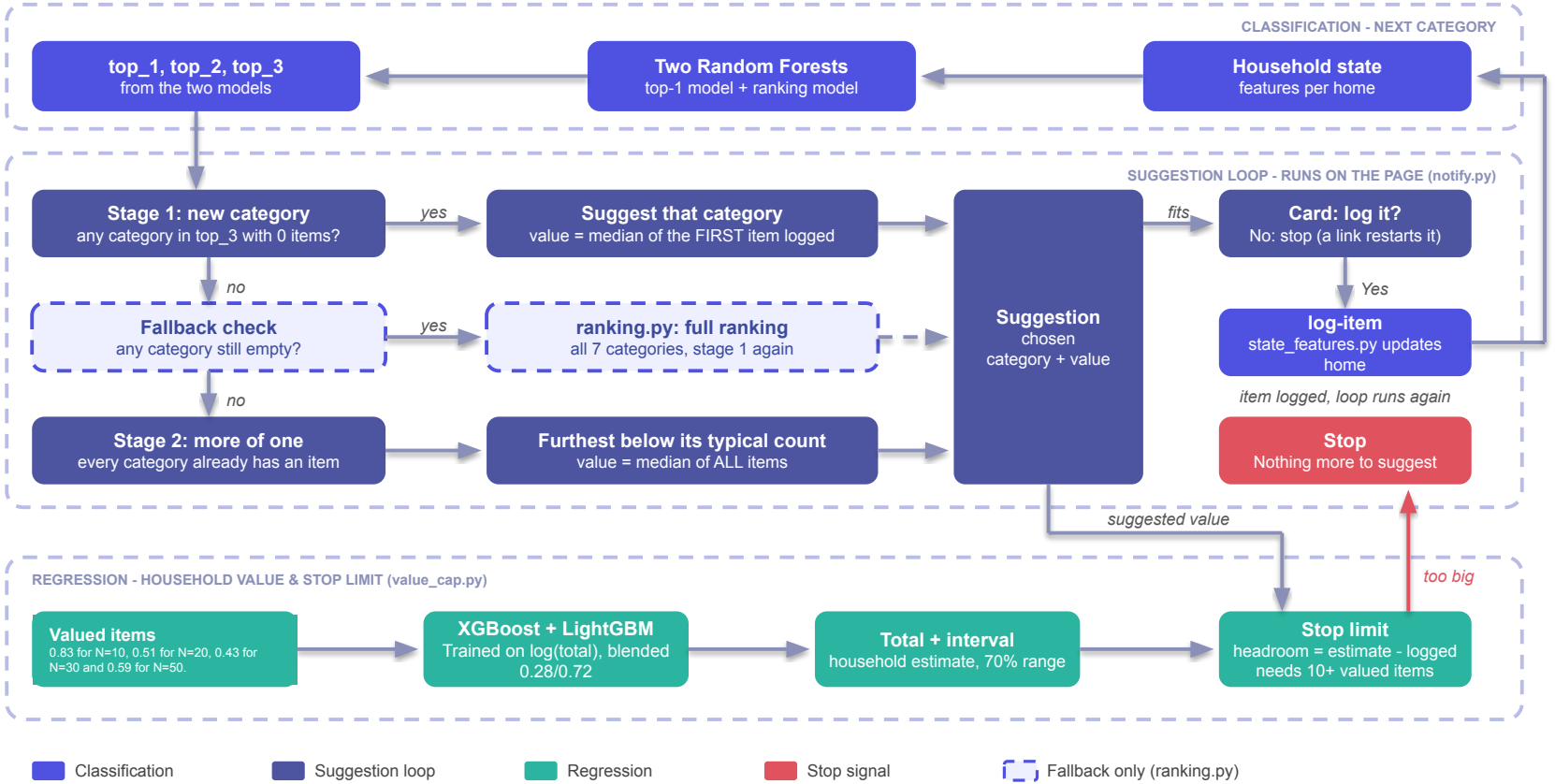
No property appears in more than one set, so the model is never tested on a home it has already seen.

Target class share



Vehicles is so rare the model barely learns it. Bars show the share of the 431 states.

Three subsystems, one page, one loop



3 · CLASSIFIER DESIGN

Two prediction objectives, two Random Forests

Objective	Model used	Why this one
Primary single guess (Top-1)	Standard Random Forest	Strongest Top-1 validation accuracy (0.4776) of any candidate - used for the headline category
Ranked shortlist (Top-2 / Top-3)	Regularised Random Forest	Smaller train/validation generalisation gap and stronger ranking behaviour (Top-3 0.8358 vs 0.8060) - used for the shortlist bars and the suggestion

Why not just pick one model?

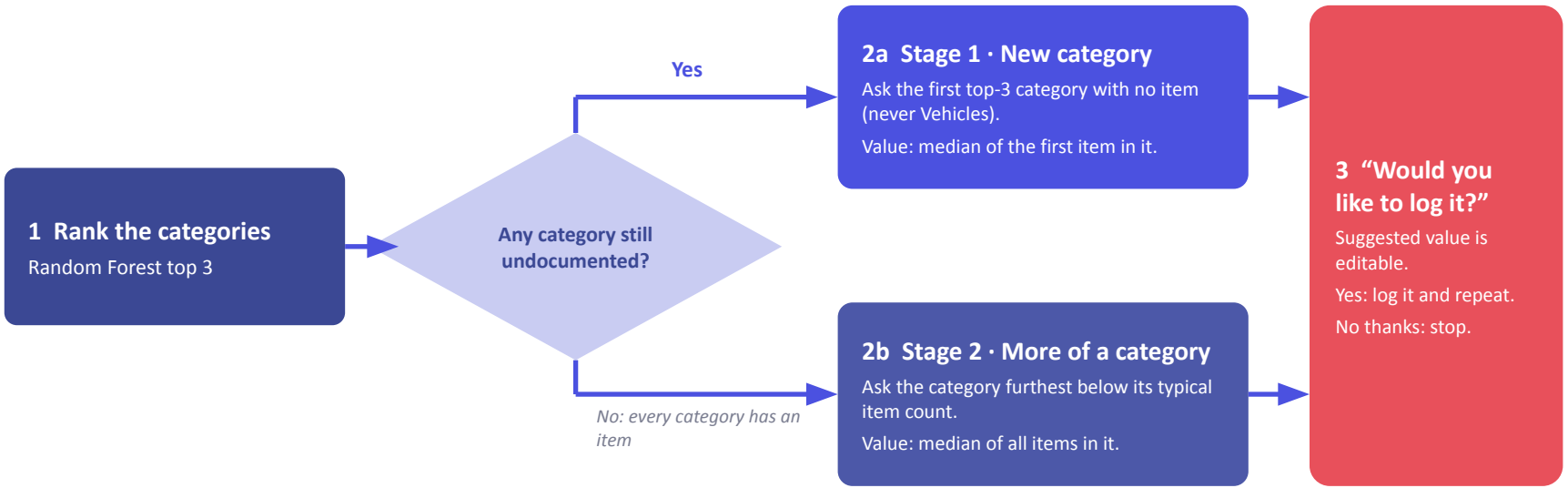
The headline and the shortlist are different products: one wants the single best guess, the other wants several reliable alternatives. The two rewarded different amounts of regularisation, so we kept both saved models rather than forcing one to do both jobs.

Six candidates were compared

- Majority-class baseline
- Logistic Regression
- Random Forest
- CatBoost (tuned)
- CatBoost (regularised)
- Random Forest (regularised)

All six on the same grouped split, same validation set - the baseline chart and full table are in Results and the appendix.

Turning a ranking into a yes/no conversation



Fallback Stage 1 reads the top 3 first, and the full ranking only if none of them can be asked while a category is empty. Without it, 1 of 431 real states jumped to stage 2 too early.

Worked example, Clothings Stage 1 suggests A$200 (first item), Stage 2 only A$37 (all items). People log their best items first, so each stage needs its own median.

Estimating when to stop asking

The household-value regression

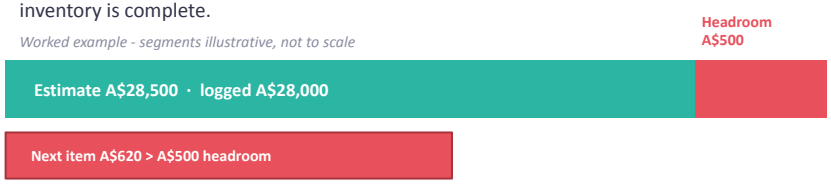
Predicts the household's total non-vehicle contents value from a basket of the items documented so far (10, 20, 30 or 50 items). **XGBoost and LightGBM are blended (weight 0.28 / 0.72)**, and a 70% interval is learned per basket size. This model was already trained and validated earlier in the project - our contribution was wiring it into the live loop as a stopping heuristic.

The stop limit

headroom = regression estimate - value already logged (never below 0)

If the next suggested value is larger than the headroom, the card is replaced by "Nothing more to suggest." A heuristic on an uncertain point estimate - not a guarantee the inventory is complete.

Worked example - segments illustrative, not to scale



→ *exceeds headroom: the loop stops, "Nothing more to suggest."*

159 / 795
properties with 10+ valued items - the regression's reach

1,840
basket rows (5 random draws per household per basket size)

16%
of real states (71 of 431) have 10+ valued items today

No limit
below 10 valued items - the loop behaves as before, on error too

A limit we're upfront about: the 70% interval is wide (about ±51% at 20 items) because the margins come from only 18 validation households, two of which set most of the width - so the point estimate can end the loop early for some households. A sensitivity check against an upper-bound gate is future work, not shipped code.

Two Random Forests: first pick right ~5 in 10, Top-3 shortlist ~8 in 10

VALIDATION (n = 67 rows) · used to choose the models

	First pick (Top-1) (F1 and accuracy)		Right answer in shortlist (accuracy)		Overfitting (Macro-F1)
	Macro-F1	Accuracy	Top-2	Top-3	Train − val
Random Forest	**0.4205**	**0.4776**	0.6567	0.8060	0.5795
RF (regularised)	0.3676	0.4328	**0.7164**	**0.8358**	0.4371
CatBoost (regularised)	0.3384	0.4328	0.7015	0.8060	**0.3605**
Logistic Regression	0.3353	0.3582	0.5672	0.8060	0.5997
CatBoost (tuned)	0.3261	0.4478	0.7015	0.8209	0.5403
Majority class	0.0610	0.2239	–	–	–

How to read this table

- Macro-F1 and Accuracy: the first pick only.
- Top-2 / Top-3: the right category is inside the shortlist (accuracy, not F1).
- Train − val: Macro-F1 on training minus validation (Top-1 only); bigger = more overfitting.
- Two models kept: Random Forest = best first pick; RF (regularised) = best shortlist, smaller drop (0.44 vs 0.58).

FINAL TEST (n = 55 rows) · untouched, scored once

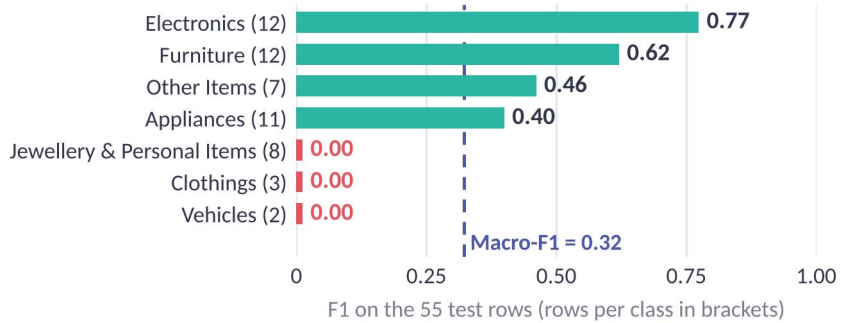
0.5091
Top-1 accuracy
28 of 55 test rows correct

0.3223
Top-1 Macro-F1
average of the 7 class F1 scores

70.9%
Top-2 accuracy (ranking model)
39 of 55 test rows

83.6%
Top-3 accuracy (ranking model)
46 of 55 test rows

Per-class F1 on the test set



Source: notebooks/03_fm_modelling.ipynb · validation: 'MODEL COMPARISON', 'UPDATED MODEL RESULTS' · test: 'FINAL HOLDOUT TEST RESULTS', 'FINAL TOP-1 CLASSIFICATION REPORT'. 1 test row = 1.8 percentage points.

Typical miss ~$19.5k | 86% hit rate fits the 70% target but is not proof

HEADLINE NUMBERS · test set = 13 households, 150 rows

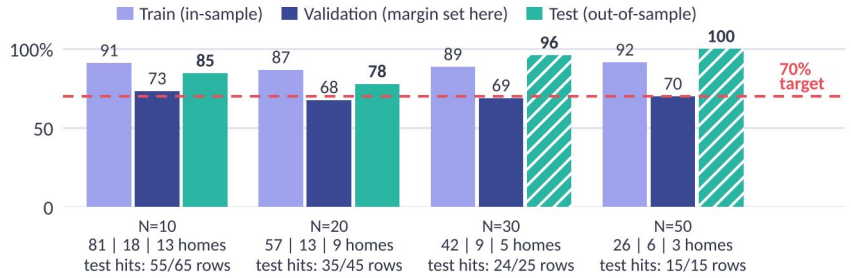
$19,544
Test MAE
150 test rows, 13 homes

87.6%
Household hit rate
true value inside the 70% interval

86.0%
Row hit rate
129 of 150 test rows

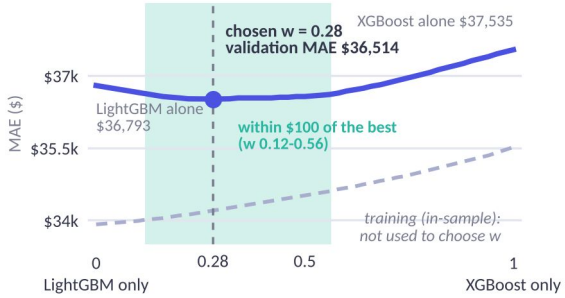
69.5%
Best estimate of real hit rate
all 31 held-out homes · 95%: 56-82%

Hit rate of the 70% interval by basket size (N = items documented)



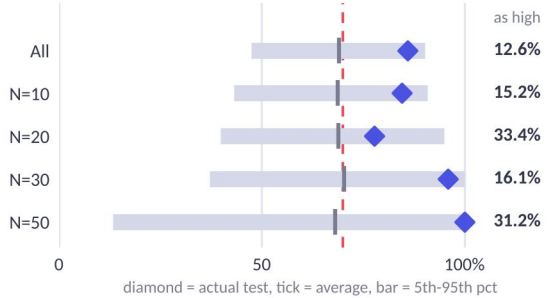
Hatched = 5 or fewer test homes: one home moves the rate by 20 points (N=30) or 33 (N=50). Homes under each N: train | validation | test. Dashed = 70% design target, not a guarantee.

Why a 0.28 / 0.72 blend?



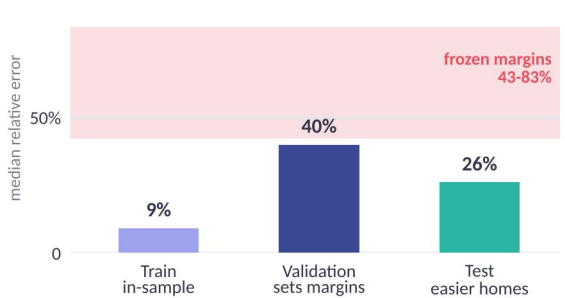
Flat curve: any weight from 0.12 to 0.56 is within $100 of the best.

Is 86% luck? 2,000 random re-splits



Re-splits average 69.1%; 86% is matched or beaten in 12.6% of them.

Why do the three bars differ?



Train is in-sample; validation sets the margins; test homes are easier.

Source: 05_ky_test_evaluation (test hit rates); 04_ky_ensemble_and_intervals (validation, blend, margins); 06_ky_split_and_sampling_audit (re-splits, error by set). Full-size versions: backup slides.

How the loop performs in replay on 71 held-out properties

HEADLINE NUMBERS · replay of 71 held-out properties

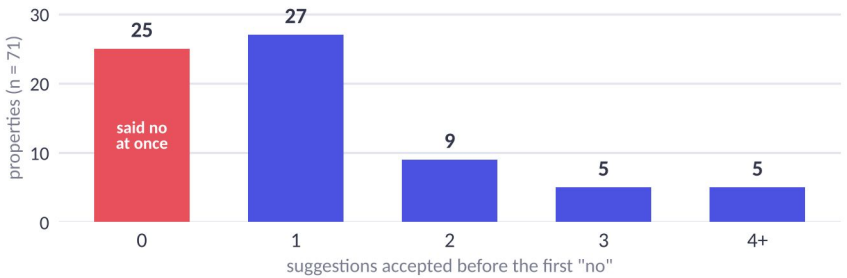
64.8%
First suggestion accepted
46 of 71 properties say "yes"

1.37 / 1
Accepted before the first "no"
mean / median

25 of 71
Said "no" at once
nothing accepted

71
Held-out properties replayed
35 validation + 36 test · property = household

How many suggestions each property accepted before its first "no"



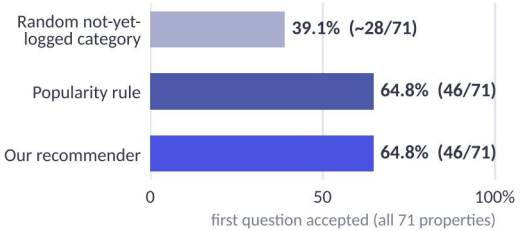
97 "yes" in total: 77 for a new category, 20 for more of a category.

One property, step by step (PRP-000750) · "yes" = its full real records contain the category

Start	The property has 1 item logged (Jewellery)		
Q1	Electronics, about A$599?	YES	real records: 3 Electronics
Q2	Furniture, about A$425?	YES	real records: 1 Furniture
Q3	Appliances, about A$800?	NO	real records: none → the loop stops

Accepted = 2: this property is one of the 9 in the "2" bar above.

First question vs simple rules

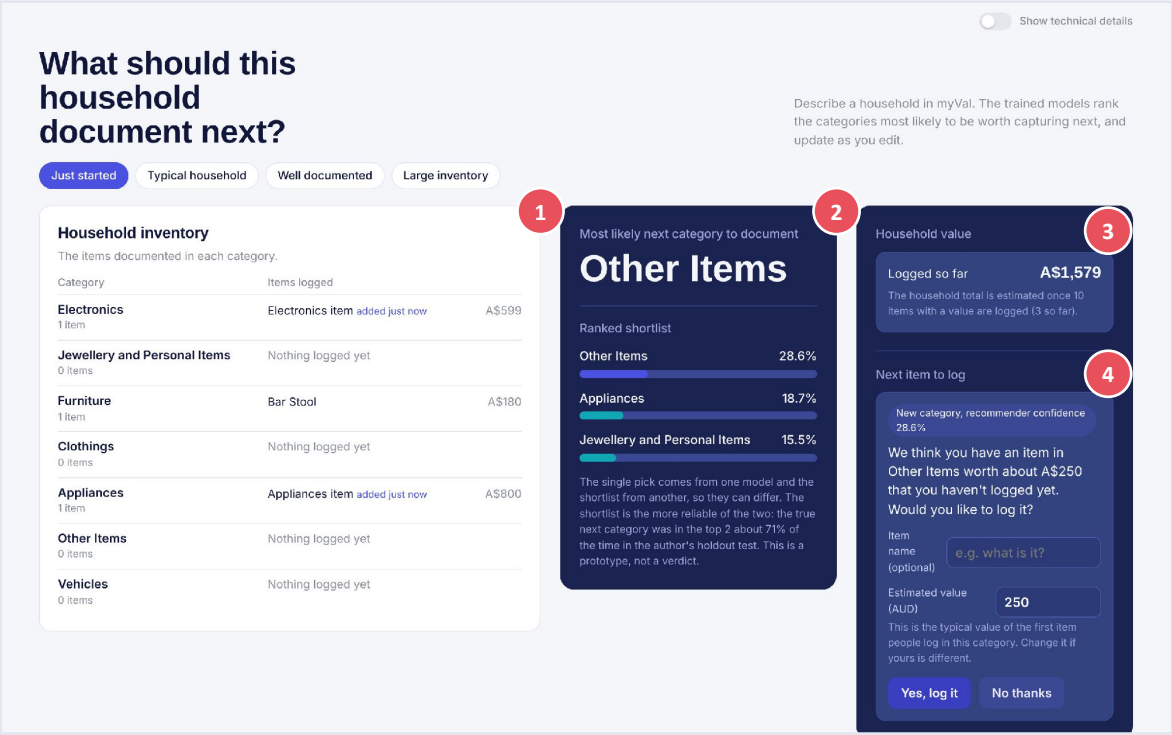


A simple popularity rule scores the same 46 of 71: the loop works, but this does not show the classifier beats it.

What this is, and isn't: a simulation with a rule-based customer, not a test with real users, so 64.8% is not model accuracy. The replay is slightly simpler than the live app (top-3 logic only).

Source: README "How well does the loop work on real households?" (our re-run gives identical numbers). Simple-rule check: our own, slide14_baseline_check.py. Details: backup slide 22.

Each “Yes” in the app refreshes the next question and the running total



1 Household inventory

What is logged so far, by category. Each “Yes” adds an item here.

2 Most likely next category

The best category not yet logged. The bars show the model’s top 3, which can include a category already logged, the app skips those.

3 Household value

“Logged so far” is the sum of the items logged. A household estimate (slide 13) appears once 10 items with a value are logged.

4 Next item to log

One question at a time, with a typical value the customer can edit. “Yes” adds it and refreshes everything, “No thanks” stops.

Source: running app (server.py + index.html), example household PRP-000059 with 3 items logged (screenshot cropped to the top part of the page).

The risks that matter most for deployment

Limitation	Evidence	Consequence	Mitigation
Small, imbalanced dataset	431 states; Vehicles has 2 test examples, Clothings 3	Vehicles F1 = 0.00; three classes never predicted on holdout	Treated as a known floor, not hidden; per-class table shown openly
AI / video features not strictly point-in-time	100/118 video rows post-date the state they describe	Some signal could leak from the future	Reviewer's refit without these groups scored close to the full model (supplementary check)
Regression retrain is not reproducible	Rerunning notebooks 02_ky–05_ky gives weight 1.0 and MAE $21,075, not 0.28 / $19,544	The committed model can't be regenerated from the pinned pipeline today	Used unchanged, exactly as committed; the discrepancy is stated, not papered over
Stage 2 is an unvalidated heuristic	Never fires on real data; proxy test on 122 states: 70.5% vs 76.2% trivial baseline	Stage 2 does not beat simply always picking the most-below-typical category	Clearly labelled a rule, not a model, in every slide and in the code comments
Stop limit covers few households today	Only 71/431 (16%) real states have 10+ valued items; 70% interval is ~±51% at N=20	Most households get no stop limit yet; estimate is imprecise where it does apply	No limit applied below the threshold or on error - loop behaves as before, never blocks

6 · CONCLUSION & REFLECTION

What we built, and what we'd do next

Technical contributions

- **Classification**: two Random Forests rank which of the six contents categories a household should document next. On the untouched test set (55 rows) the first pick is right 51% of the time (Top-1 0.509) and the right category is in the top 3 for 84% (46 of 55).
- **Regression**: an XGBoost and LightGBM blend estimates a household's total contents value from the first 10 to 50 items logged. Test MAE is $19,544 on 13 homes; the 70% interval's 86% hit rate fits its target but is not proof of calibration.
- A live two-stage suggestion loop on top of the classifier, with an editable median-based value per item
- A regression-backed stop limit (headroom = estimate − logged) wired into every request

Next steps

- Validate Stage 2 once more households document all six contents categories
- Collect more valued-item households so the stop limit reaches past today's 16% of states.
- Retrain the models on real datasets to check if model's metrics might improve.

Reflection: the honest result here isn't just the 64.8% simulated acceptance, it's that real held-out data and a real running app let us say exactly where the design is strong (the median-over-model finding, the AI-feature tracking, the real screenshots you just saw) and exactly where it still rests on an untested assumption (Stage 2, replay vs. real acceptance). Shipping that distinction clearly is the actual deliverable for a technical review.



Thank you